🎯 GLA UNIVERSITY — DAY 2 PRE-ASSESSMENT 50 MCQS (OFFICIAL INSTRUCTOR MASTER)

Traditional Machine Learning & Parallel Computing using Intel oneAPI (`GLA-HPC-BCSG701`)

Lead Trainer: Shivam Gupta | **Target Session:** Day 2 Start (Diagnostic Baseline Assessment)
Total Questions: 50 Multi-Choice Questions | **Target Diagnostic Score:** 40% – 60% Baseline
Evaluation Specifications: Official 7-Column University Examination Spec

---

📌 Pedagogical Architecture & Diagnostic Baseline Strategy

Administered at the start of Day 2 (09:00 AM – 09:30 AM), this 50-question assessment gauges incoming competence across:

- **Section A (Q01 – Q15):** Prerequisite OS Concurrency & Memory (Processes vs Threads, Mutexes, Deadlocks, Context Switching, Cache Line Alignment)
- **Section B (Q16 – Q30):** Prerequisite Python & Scikit-Learn ML Foundations (Supervised vs Unsupervised, KMeans, PCA, SVM, Random Forest, Scaling, GIL)
- **Section C (Q31 – Q40):** Applied Parallel Concepts & Day 1 Previews (MIMD/SIMD, Strong/Weak Scaling, OpenMP vs MPI memory, Hybrid Model, Loop Dependencies)
- **Section D (Q41 – Q50):** Day 2 Advanced Hardware & Toolkit Topics Preview (oneDAL scope, sklearnex, oneMKL BLAS, Modin, False Sharing, Tree Collectives)

Question 01 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | Foundational Prerequisite]

Topic: Process vs Thread Memory Spaces
Question: What is the primary difference between a process and a thread regarding their virtual memory address spaces in modern operating systems?

- **a)** Processes share heap memory by default, whereas threads have isolated heaps
- **b)** Threads within the same process share code, data, and heap segments but possess their own private stack and register state; processes have strictly isolated virtual address spaces
- **c)** Threads cannot execute CPU instructions simultaneously
- **d)** Processes do not require page tables

> **TIP**
> **Correct Answer: [b]** — Threads within the same process share code, data, and heap segments but possess their own private stack and register state; processes have strictly isolated virtual address spaces
> **Technical Explanation:** Threads created within a single process share the text, data, and heap segments, enabling fast shared-memory communication. However, each thread maintains an independent program counter, registers, and private stack frame. In contrast, separate processes have completely isolated address spaces guarded by hardware MMU page tables.

Question 02 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | Foundational Prerequisite]

Topic: Race Conditions & Mutual Exclusion
Question: When multiple threads concurrently execute `counter = counter + 1` without synchronization, why can the final value be smaller than expected?

- **a)** The CPU drops floating-point numbers
- **b)** The operation consists of three assembly instructions (Read, Modify, Write) that can be interleaved arbitrarily across threads, resulting in lost updates
- **c)** The OS scheduler deletes variables that grow too fast
- **d)** Threads run at different clock voltages

> **TIP**
> **Correct Answer: [b]** — The operation consists of three assembly instructions (Read, Modify, Write) that can be interleaved arbitrarily across threads, resulting in lost updates
> **Technical Explanation:** `counter++` is not an atomic operation. It translates to: `mov eax, [counter]; add eax, 1; mov [counter], eax`. If Thread A reads 5, and Thread B reads 5 before Thread A writes back 6, both threads write 6, losing one increment.

Question 03 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | Foundational Prerequisite]

Topic: Critical Sections & Mutex Locks
Question: What is the primary function of a Mutex (Mutual Exclusion Lock) in multi-threaded programming?

- **a)** To accelerate thread execution by 200%
- **b)** To ensure that only one thread can execute a critical section of code at any given time, blocking other threads until the lock is released
- **c)** To convert Python functions into assembly code
- **d)** To distribute network traffic across cluster switches

> **TIP**
> **Correct Answer: [b]** — To ensure that only one thread can execute a critical section of code at any given time, blocking other threads until the lock is released
> **Technical Explanation:** A mutex provides mutual exclusion by enforcing that only the thread acquiring the lock enters the critical section. Any other thread attempting to acquire the locked mutex is placed into a blocked/waiting state until the owning thread unlocks it.

Question 04 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Foundational Prerequisite]

Topic: Deadlock Coffman Conditions
Question: Which of the following is NOT one of the four necessary Coffman conditions required for a system deadlock to occur?

- **a)** Mutual Exclusion
- **b)** Hold and Wait
- **c)** Preemption allowed by default
- **d)** Circular Wait

> **TIP**
> **Correct Answer: [c]** — Preemption allowed by default
> **Technical Explanation:** The four Coffman conditions are: (1) Mutual Exclusion, (2) Hold and Wait, (3) No Preemption (resources cannot be forcibly confiscated from a thread holding them), and (4) Circular Wait. Allowing preemption prevents deadlocks.

Question 05 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Foundational Prerequisite]

Topic: Context Switching Overhead
Question: Why is switching between two threads of the SAME process generally faster than switching between two threads of DIFFERENT processes?

- **a)** Same-process threads run on the same frequency
- **b)** Switching threads within the same process avoids flushing the Translation Lookaside Buffer (TLB) and switching the page table base register (CR3 on x86)
- **c)** Different processes must write their memory to hard disk before switching
- **d)** Same-process threads do not save CPU registers

> **TIP**
> **Correct Answer: [b]** — Switching threads within the same process avoids flushing the Translation Lookaside Buffer (TLB) and switching the page table base register (CR3 on x86)
> **Technical Explanation:** Threads within the same process share page tables. Switching between them only requires saving and restoring register state and stack pointers. Switching across processes changes the virtual memory map (CR3 register), which invalidates non-global TLB cache entries, causing severe memory latency.

Question 06 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Foundational Prerequisite]

Topic: Spinlocks vs Blocking Mutexes
Question: Under what specific condition is a Spinlock preferable over a standard blocking OS Mutex in high-performance computing?

- **a)** When the critical section is very long (takes several minutes)

- **b)** When the lock is held for an extremely short duration (a few nanoseconds/cycles) and the cost of an OS context switch exceeds the cost of busy-waiting
- **c)** When running on single-core uniprocessor systems
- **d)** When writing data to remote tape storage

> **TIP**
> **Correct Answer: [b]** — When the lock is held for an extremely short duration (a few nanoseconds/cycles) and the cost of an OS context switch exceeds the cost of busy-waiting
> **Technical Explanation:** An OS mutex context switch incurs thousands of CPU cycles in kernel transitions and scheduler overhead. If a critical section is only a few cycles long, a spinlock busy-waits in user space on a multi-core CPU, avoiding expensive kernel sleep/wakeup transitions.

Question 07 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Hardware Architecture]

Topic: Memory Alignment & Hardware Cache Lines
Question: What is the typical physical size of an L1 Data Cache line on modern Intel Xeon and x86_64 processors?

- **a)** 8 bytes
- **b)** 64 bytes
- **c)** 512 bytes
- **d)** 4096 bytes

> **TIP**
> **Correct Answer: [b]** — 64 bytes
> **Technical Explanation:** Standard modern x86_64 and Intel Xeon architectures utilize 64-byte cache lines. Whenever a single byte is requested, the hardware memory controller fetches the entire contiguous 64-byte aligned block from memory into cache.

Question 08 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Memory Cache Locality]

Topic: Row-Major vs Column-Major Matrix Traversal
Question: In C and C++, 2D arrays are stored in Row-Major order in memory. Which nested loop traversal yields the highest cache hit rate for an $N \times N$ matrix?

- **a)** `for (j=0; j<N; j++) for (i=0; i<N; i++) A[i][j] = 0;` (Column-by-Column)
- **b)** `for (i=0; i<N; i++) for (j=0; j<N; j++) A[i][j] = 0;` (Row-by-Row)
- **c)** Both loops have identical cache hit rates
- **d)** Random index hopping

> **TIP**
> **Correct Answer: [b]** — `for (i=0; i<N; i++) for (j=0; j<N; j++) A[i][j] = 0;` (Row-by-Row)
> **Technical Explanation:** Row-major order stores elements of the same row sequentially in adjacent memory addresses. Looping `A[i][j]` over `j` accesses contiguous memory words within the same 64-byte cache line (spatial locality). Looping over `i` strided by row size causes a cache miss on every access.

Question 09 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Compiler Semantics]

Topic: Volatile Keyword in C/C++
Question: What does declaring a variable `volatile int flag;` instruct the C/C++ compiler to do during optimization?

- **a)** It makes all operations on `flag` atomic across threads
- **b)** It prevents the compiler from caching `flag` in a CPU register, forcing every read and write to access physical memory/cache directly
- **c)** It places the variable in the GPU memory space
- **d)** It automatically encrypts the variable value

> **TIP**
> **Correct Answer: [b]** — It prevents the compiler from caching `flag` in a CPU register, forcing every read and write to access physical memory/cache directly
> **Technical Explanation:** `volatile` tells the compiler that the variable may change unexpectedly (e.g., via hardware or another thread), so it must not optimize away reads or cache the variable in a register across loop iterations. However, `volatile` does NOT provide atomic synchronization or memory barriers.

Question 10 [Prerequisite Foundations: Operating Systems & Concurrency | Hard | Hardware Concurrency]

Topic: Memory Barriers (Fences)
Question: Why are Memory Barriers (fences) necessary when implementing lock-free data structures on modern out-of-order execution CPUs?

- **a)** To stop the CPU fan from spinning too fast
- **b)** To enforce strict memory ordering, preventing hardware CPU pipelines and optimizing compilers from reordering reads and writes across the barrier boundary
- **c)** To convert 32-bit pointers to 64-bit pointers
- **d)** To force the operating system to shut down idle cores

> **TIP**
> **Correct Answer: [b]** — To enforce strict memory ordering, preventing hardware CPU pipelines and optimizing compilers from reordering reads and writes across the barrier boundary
> **Technical Explanation:** Modern superscalar processors reorder memory instructions to maximize pipeline utilization (e.g., Store-Load reordering). A memory barrier (`mfence` , `atomic_thread_fence`) enforces that memory operations initiated before the barrier become globally visible before operations after the barrier.

Question 11 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | OS Foundations]

Topic: Inter-Process Communication (IPC) Mechanisms
Question: Which IPC mechanism on a single Linux host provides the highest data transfer bandwidth by eliminating kernel data copying?

- **a)** UNIX domain sockets
- **b)** POSIX Shared Memory (`shm_open` / `mmap`)

- **c)** Anonymous Pipes (`pipe()`)
- **d)** Standard TCP loopback sockets (`127.0.0.1`)

> **TIP**
> **Correct Answer: [b]** — POSIX Shared Memory (`shm_open` / `mmap`)
> **Technical Explanation:** POSIX Shared Memory maps the exact same physical RAM pages into the virtual address spaces of both processes. Once established, processes communicate at raw memory bus speeds with zero system call overhead and zero kernel buffer copying.

Question 12 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | Systems Programming]

Topic: POSIX Thread Creation (`pthread_create`)
Question: In C POSIX threads (`pthread`), which function is called by a parent thread to block until a specific target child thread has completed execution?

- **a)** `pthread_yield()`
- **b)** `pthread_join()`
- **c)** `pthread_exit()`
- **d)** `pthread_detach()`

> **TIP**
> **Correct Answer: [b]** — `pthread_join()`
> **Technical Explanation:** `pthread_join(thread, &retval)` suspends execution of the calling thread until the target thread terminates, releasing the child thread's resources and collecting its return status.

Question 13 [Prerequisite Foundations: Operating Systems & Concurrency | Medium | Software Architecture]

Topic: Thread Pool Architecture
Question: Why do high-performance servers and OpenMP runtimes utilize a pre-allocated 'Thread Pool' rather than creating a new thread for every incoming task?

- **a)** Because operating systems can only create a maximum of 4 threads in their lifetime
- **b)** To eliminate the substantial kernel latency of repeatedly allocating thread stacks, creating kernel scheduling entities, and deallocating thread resources on every request
- **c)** To ensure that all tasks execute in alphabetical order
- **d)** Because thread pools eliminate the need for RAM

> **TIP**
> **Correct Answer: [b]** — To eliminate the substantial kernel latency of repeatedly allocating thread stacks, creating kernel scheduling entities, and deallocating thread resources on every request
> **Technical Explanation:** Creating and destroying OS threads incurs thousands of cycles in stack allocation, kernel thread table updates, and context setup. A thread pool keeps worker threads alive in a waiting queue, assigning tasks with near-zero scheduling overhead.

Question 14 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | Software Architecture]

Topic: Thread Safety Definition
Question: What does it mean for a C++ library function (e.g., in a math library) to be 'Thread-Safe'?

- **a)** It can only be executed by one single thread across the entire program runtime
- **b)** It can be invoked simultaneously by multiple threads without causing race conditions, memory corruption, or undefined behavior
- **c)** It automatically disables multi-threading in the calling program
- **d)** It requires root permissions to execute

> **TIP**
> **Correct Answer: [b]** — It can be invoked simultaneously by multiple threads without causing race conditions, memory corruption, or undefined behavior
> **Technical Explanation:** A thread-safe function guarantees correct execution when called concurrently by multiple threads, typically by avoiding mutable static/global state, using thread-local storage, or synchronizing shared resource access.

Question 15 [Prerequisite Foundations: Operating Systems & Concurrency | Easy | Parallel Mathematics]

Topic: Amdahl's Law Fundamental Equation
Question: What is the classical formula for Amdahl's Law calculating speedup $S(N)$ for a parallel fraction P and sequential fraction $S = 1 - P$ on N processors?

- **a)** $S(N) = N \times P$
- **b)** $S(N) = \frac{1}{S + \frac{P}{N}}$
- **c)** $S(N) = \frac{1}{N} + P$
- **d)** $S(N) = P - S \times N$

> **TIP**
> **Correct Answer: [b]** — $S(N) = \frac{1}{S + \frac{P}{N}}$
> **Technical Explanation:** Amdahl's Law states: Speedup = 1 / [S + (P / N)], where S is the serial fraction and P is the parallelizable fraction across N processing units.

Question 16 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Easy | ML Theory]

Topic: Supervised vs Unsupervised Learning
Question: What is the fundamental distinguishing characteristic of Supervised Learning compared to Unsupervised Learning?

- **a)** Supervised learning requires GPU acceleration, while unsupervised does not
- **b)** Supervised learning trains on labeled input-output pairs (X, y) to learn a mapping function; Unsupervised learning discovers latent patterns and clusters within unlabeled data X without ground-truth targets
- **c)** Supervised learning only works on images, while unsupervised works on text
- **d)** Unsupervised learning always achieves 100% accuracy

> **TIP**
>
> **Correct Answer: [b]** — Supervised learning trains on labeled input-output pairs (X, y) to learn a mapping function; Unsupervised learning discovers latent patterns and clusters within unlabeled data X without ground-truth targets
> **Technical Explanation:** Supervised learning uses target labels (y) to guide loss optimization (e.g., classification, regression). Unsupervised learning operates strictly on feature vectors (X) to identify intrinsic structures, densities, or groupings (e.g., K-Means, PCA).

Question 17 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | ML Algorithms]

Topic: K-Means Clustering Objective Function
Question: What mathematical objective function does the classical K-Means algorithm minimize during iterative centroid updates?

- **a)** Cross-entropy loss
- **b)** Within-Cluster Sum of Squares (WCSS) / Inertia, measuring the sum of squared Euclidean distances between points and their assigned cluster centroids
- **c)** Hinge loss
- **d)** Maximum likelihood of Markov chains

> **TIP**
>
> **Correct Answer: [b]** — Within-Cluster Sum of Squares (WCSS) / Inertia, measuring the sum of squared Euclidean distances between points and their assigned cluster centroids
> **Technical Explanation:** K-Means minimizes inertia (WCSS): $\sum_{i=1}^{k} \sum_{x \in S_i} ||x - \mu_i||^2$, where μ_i is the centroid of cluster S_i. Each iteration assigns points to nearest centroids and recalculates centroids as cluster means.

Question 18 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | Dimensionality Reduction]

Topic: Principal Component Analysis (PCA)
Question: What is the mathematical goal of Principal Component Analysis (PCA) in machine learning preprocessing?

- **a)** To convert tabular data into audio waveforms
- **b)** To project high-dimensional data onto orthogonal axes (eigenvectors of the covariance matrix) that maximize the variance of the data along the principal dimensions
- **c)** To eliminate rows with negative numbers
- **d)** To encrypt user data for privacy compliance

> **TIP**
>
> **Correct Answer: [b]** — To project high-dimensional data onto orthogonal axes (eigenvectors of the covariance matrix) that maximize the variance of the data along the principal dimensions
> **Technical Explanation:** PCA performs an orthogonal linear transformation that maps data to a coordinate system where the greatest variance lies on the first coordinate (first principal component), the second greatest variance on the second, and so on, via Singular Value Decomposition (SVD).

Question 19 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | ML Algorithms]

Topic: Support Vector Machine (SVM) Hyperplane
Question: In a linear Support Vector Classifier (SVC), what defines the 'optimal margin' separating two classes?

- **a)** The line that connects the average of all points in both classes
- **b)** The decision boundary that maximizes the geometric distance (margin) between the separating hyperplane and the closest data points (support vectors) of either class
- **c)** The curve that passes through every single data point with zero residual
- **d)** The boundary chosen randomly by gradient descent

> **TIP**
> **Correct Answer: [b]** — The decision boundary that maximizes the geometric distance (margin) between the separating hyperplane and the closest data points (support vectors) of either class
> **Technical Explanation:** SVM aims to find the maximum-margin hyperplane: $\max \frac{2}{||w||}$ subject to $y_i(w \cdot x_i + b) \geq 1$. Maximizing the margin between support vectors yields the best theoretical generalization on unseen test data.

Question 20 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Easy | Ensemble Methods]

Topic: Random Forest Ensemble Mechanism
Question: How does a Random Forest Classifier improve predictive accuracy and prevent overfitting compared to a single Decision Tree?

- **a)** By running decision trees in reverse order
- **b)** By training an ensemble of de-correlated decision trees using bootstrap aggregating (bagging) and random feature subsets, then averaging their predictions (majority voting)
- **c)** By deleting all trees that have more than 3 branches
- **d)** By converting all input features into binary zeros and ones

> **TIP**
> **Correct Answer: [b]** — By training an ensemble of de-correlated decision trees using bootstrap aggregating (bagging) and random feature subsets, then averaging their predictions (majority voting)
> **Technical Explanation:** A single decision tree has high variance and tends to overfit. Random Forest trains multiple trees on bootstrap subsets of data and considers a random subset of features at each split. Averaging over independent trees reduces variance without increasing bias.

Question 21 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Easy | ML Diagnostics]

Topic: Overfitting vs Underfitting
Question: An ML model achieves 99.8% training accuracy on a cluster dataset but drops to 52.1% accuracy on the independent test validation set. What is this phenomenon called?

- **a)** Underfitting (High Bias)
- **b)** Overfitting (High Variance)
- **c)** Optimal Generalization

- **d)** Data Quantization

> **TIP**
> **Correct Answer: [b]** — Overfitting (High Variance)
> **Technical Explanation:** Overfitting occurs when a model learns the statistical noise and idiosyncrasies of the training set rather than the underlying general trend, yielding near-perfect training scores but catastrophic failure on unseen test data.

Question 22 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | Preprocessing]

Topic: Feature Scaling (StandardScaler)
Question: Why is feature standardization (`StandardScaler` in Scikit-Learn) mandatory before training distance-based algorithms like K-Means, KNN, and SVM with RBF kernels?

- **a)** Because Python cannot divide by numbers greater than 100
- **b)** Because features with large numerical magnitudes (e.g., annual income in thousands) would dominate Euclidean distance calculations over features with small scales (e.g., age in years)
- **c)** To convert continuous numbers into text strings
- **d)** StandardScaler is only used to format plots for publication

> **TIP**
> **Correct Answer: [b]** — Because features with large numerical magnitudes (e.g., annual income in thousands) would dominate Euclidean distance calculations over features with small scales (e.g., age in years)
> **Technical Explanation:** Distance-based metrics ($||x_i - x_j|| = \sqrt{\sum(x_{ik} - x_{jk})^2}$) are sensitive to feature scales. If one feature spans 0 to 1,000,000 and another spans 0 to 1, the first feature completely dictates distances. `StandardScaler` centers data to mean 0 and unit variance 1.

Question 23 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Easy | Scikit-Learn Architecture]

Topic: Scikit-Learn Estimator API Consistency
Question: In Scikit-Learn's object-oriented API design, which method is consistently implemented across all estimator algorithms to execute model training?

- **a)** `model.train()`
- **b)** `model.fit(X, y)`
- **c)** `model.learn()`
- **d)** `model.execute()`

> **TIP**
> **Correct Answer: [b]** — `model.fit(X, y)`
> **Technical Explanation:** Scikit-learn strictly standardizes on the Estimator API: `fit(X, y)` trains the model, `predict(X)` generates predictions, and `transform(X)` applies learned pre-processing transformations.

Question 24 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Hard | NumPy Internals]

Topic: NumPy Array Contiguity (`C_CONTIGUOUS`)
Question: Why do high-performance C/C++ libraries (like oneDAL and BLAS) execute significantly faster on `C_CONTIGUOUS` NumPy arrays compared to non-contiguous sliced arrays?

- **a)** Non-contiguous arrays can only hold integer numbers
- **b)** `C_CONTIGUOUS` arrays reside in a single flat block of linear physical memory where consecutive row elements are adjacent, enabling direct vector SIMD memory loads without pointer indirection or memory copying
- **c)** NumPy non-contiguous arrays are stored on hard disk instead of RAM
- **d)** Python prohibits non-contiguous array indexing

> **TIP**
> **Correct Answer: [b]** — `C_CONTIGUOUS` arrays reside in a single flat block of linear physical memory where consecutive row elements are adjacent, enabling direct vector SIMD memory loads without pointer indirection or memory copying
> **Technical Explanation:** A C-contiguous array has elements laid out sequentially in row-major order with stride matching element byte size. SIMD load instructions (`_mm512_loadu_ps`) require contiguous memory addresses. Non-contiguous arrays force element-by-element gather operations or memory allocation copies.

Question 25 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | Python Runtime]

Topic: Python Global Interpreter Lock (GIL)
Question: How does the Python Global Interpreter Lock (GIL) impact pure Python multi-threaded CPU-bound programs on multi-core servers?

- **a)** It doubles the speed of Python loops on multiple cores
- **b)** It prevents multiple OS threads from executing Python bytecode simultaneously, effectively serializing execution onto a single CPU core despite having multiple threads
- **c)** It forces Python to run inside a web browser
- **d)** It automatically converts Python to C++ code

> **TIP**
> **Correct Answer: [b]** — It prevents multiple OS threads from executing Python bytecode simultaneously, effectively serializing execution onto a single CPU core despite having multiple threads
> **Technical Explanation:** The CPython GIL is a mutual exclusion lock used by the interpreter to protect internal object reference counts. As a consequence, even if 16 Python threads are active on 16 cores, only ONE thread can execute Python bytecode at any given moment, bottlenecking CPU-bound code.

Question 26 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | HPC Python Optimization]

Topic: Bypassing the GIL in NumPy & C Extensions
Question: Why do NumPy vector operations (e.g., `np.dot(A, B)`) and Intel oneDAL kernels scale across multiple CPU cores in Python despite the existence of the GIL?

- **a)** NumPy runs in a separate Linux container

- **b)** C/C++ extensions and oneDAL explicitly release the Python GIL (`Py_BEGIN_ALLOW_THREADS`) before entering compute-intensive loops, allowing native threads to execute concurrently across physical cores
- **c)** NumPy deletes the GIL permanently upon installation
- **d)** NumPy only uses 1 core at all times

> **TIP**
> **Correct Answer: [b]** — C/C++ extensions and oneDAL explicitly release the Python GIL (`Py_BEGIN_ALLOW_THREADS`) before entering compute-intensive loops, allowing native threads to execute concurrently across physical cores
> **Technical Explanation:** The GIL only protects Python object evaluation. When NumPy or oneDAL calls native compiled C/C++ routines, it releases the GIL using Python C-API macros. OpenMP or native threads within oneMKL/oneDAL then run with full multi-core hardware parallelism.

Question 27 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | ML Best Practices]

Topic: Train/Test Data Leakage
Question: What fatal methodological flaw occurs if a data scientist fits `StandardScaler().fit(X)` on the ENTIRE dataset before performing `train_test_split()` ?

- **a)** The computer runs out of memory
- **b)** Data Leakage: Information from the test validation set (such as the mean and standard deviation) leaks into the training pipeline, yielding overly optimistic evaluation metrics that fail in production
- **c)** Scikit-learn raises a `SyntaxError` exception
- **d)** The test set becomes empty

> **TIP**
> **Correct Answer: [b]** — Data Leakage: Information from the test validation set (such as the mean and standard deviation) leaks into the training pipeline, yielding overly optimistic evaluation metrics that fail in production
> **Technical Explanation:** Fitting preprocessors on the entire dataset exposes the model to test distribution properties (mean, variance) prior to evaluation. Preprocessors must ALWAYS be fit strictly on the training partition (`scaler.fit(X_train)`), and applied via `transform()` to both train and test.

Question 28 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Easy | Model Evaluation]

Topic: Confusion Matrix & Evaluation Metrics
Question: In medical disease diagnosis or critical fraud detection where false negatives (missing a true positive condition) have severe consequences, which metric should be prioritized?

- **a)** Accuracy on an imbalanced dataset
- **b)** Recall (Sensitivity / True Positive Rate)
- **c)** Training Loss
- **d)** Number of features

> **TIP**
> **Correct Answer: [b]** — Recall (Sensitivity / True Positive Rate)
> **Technical Explanation:** Recall measures the proportion of actual positives that were correctly identified: $\text{Recall} = \frac{TP}{TP+FN}$. In safety-critical or disease detection tasks, minimizing False Negatives (maximizing Recall) is critical.

Question 29 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Easy | ML Algorithms]

Topic: Logistic Regression vs Linear Regression
Question: What mathematical function does Logistic Regression apply to linear combinations of inputs ($z = w^T x + b$) to map outputs to probabilities between 0 and 1?

- **a)** Rectified Linear Unit (ReLU)
- **b)** Sigmoid (Logistic) Function: $\sigma(z) = \frac{1}{1+e^{-z}}$
- **c)** Hyperbolic Tangent ($tanh$)
- **d)** Cosine similarity

> **TIP**
> **Correct Answer: [b]** — Sigmoid (Logistic) Function: $\sigma(z) = \frac{1}{1+e^{-z}}$
> **Technical Explanation:** Logistic regression passes the linear prediction through the Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, producing a value bounded between 0 and 1 that represents the class probability $P(y = 1|x)$.

Question 30 [Prerequisite Foundations: Classical Machine Learning & Scikit-Learn | Medium | Model Validation]

Topic: Cross-Validation (K-Fold)
Question: How does 5-Fold Cross-Validation provide a more robust performance estimate than a single train/test split?

- **a)** It generates synthetic data using deep generative models
- **b)** It partitions the dataset into 5 equal folds, training 5 separate models where each fold serves as the test set once while the remaining 4 serve as training, averaging the validation scores
- **c)** It runs the training loop 5 times on the exact same data to verify Python speed
- **d)** It divides the dataset into 5 separate CSV files on disk

> **TIP**
> **Correct Answer: [b]** — It partitions the dataset into 5 equal folds, training 5 separate models where each fold serves as the test set once while the remaining 4 serve as training, averaging the validation scores
> **Technical Explanation:** K-Fold cross-validation ensures every data point is used for both training and validation across the iterations. Averaging performance across all K folds drastically reduces the variance associated with a single lucky or unlucky random train/test split.

Question 31 [Applied Parallel Concepts & Architecture Preview | Medium | Architecture Review]

Topic: Flynn's Taxonomy in Heterogeneous HPC
Question: A cluster node contains an Intel Xeon CPU executing multiple independent MPI processes simultaneously, while

its vector registers execute AVX-512 vector instructions. Under Flynn's taxonomy, how is this system categorized at the node and core levels respectively?

- **a)** SISD at the node level, MISD at the core level
- **b)** MIMD (Multiple Instruction, Multiple Data) across nodes/processes, and SIMD (Single Instruction, Multiple Data) within individual vector cores
- **c)** MISD across nodes, SISD within cores
- **d)** Pure SISD exclusively throughout the cluster

> **TIP**
> **Correct Answer: [b]** — MIMD (Multiple Instruction, Multiple Data) across nodes/processes, and SIMD (Single Instruction, Multiple Data) within individual vector cores
> **Technical Explanation:** Clusters and multi-core processors operate as MIMD systems (each core executes distinct instruction streams on different data). Within each core, vector ALUs (AVX-512) execute SIMD operations (one instruction applies across a vector of data elements).

Question 32 [Applied Parallel Concepts & Architecture Preview | Medium | Performance Analysis]

Topic: Speedup vs Scalability
Question: What is the defining distinction between 'Strong Scaling' and 'Weak Scaling' in parallel performance benchmarking?

- **a)** Strong scaling uses Intel CPUs; Weak scaling uses AMD CPUs
- **b)** In Strong Scaling, the total problem size remains fixed as processor count increases; in Weak Scaling, the problem size increases proportionally with the number of processors
- **c)** Strong scaling measures GPU speed; Weak scaling measures disk speed
- **d)** Weak scaling assumes all processors run at half voltage

> **TIP**
> **Correct Answer: [b]** — In Strong Scaling, the total problem size remains fixed as processor count increases; in Weak Scaling, the problem size increases proportionally with the number of processors
> **Technical Explanation:** Strong scaling tests how quickly a fixed problem can be solved by adding processors (governed by Amdahl's Law). Weak scaling tests whether adding processors allows proportionately larger problems to be solved in constant time (governed by Gustafson's Law).

Question 33 [Applied Parallel Concepts & Architecture Preview | Medium | HPC Bottlenecks]

Topic: Distributed Cluster Memory Wall
Question: Why is inter-node network latency (e.g., Ethernet vs InfiniBand) often the dominant bottleneck in distributed MPI training rather than pure CPU FLOPS?

- **a)** Because CPUs execute code too slowly to notice the network
- **b)** Because network transfer latency (microseconds to milliseconds) is orders of magnitude slower than on-chip L1/L2 cache accesses (nanoseconds), causing processors to stall waiting for data transfers
- **c)** Because network cables compress data automatically
- **d)** Because Linux terminates network sockets every 10 seconds

> **TIP**
>
> **Correct Answer: [b]** — Because network transfer latency (microseconds to milliseconds) is orders of magnitude slower than on-chip L1/L2 cache accesses (nanoseconds), causing processors to stall waiting for data transfers
> **Technical Explanation:** CPUs access L1 cache in ~1 ns and local RAM in ~60-80 ns. In contrast, standard Ethernet latency is ~10-50 microseconds. In distributed computing, communication latency dominates if synchronization occurs frequently.

Question 34 [Applied Parallel Concepts & Architecture Preview | Easy | OpenMP Fundamentals]

Topic: OpenMP Shared Memory Scope
Question: Can OpenMP parallel directives (`#pragma omp parallel`) run across multiple separate server computers connected via network cables without MPI?

- **a)** Yes, OpenMP automatically creates a virtual cloud across computers
- **b)** No, OpenMP is strictly designed for Shared-Memory systems where threads access the same physical motherboard address space
- **c)** Yes, but only if all computers run Windows 11
- **d)** Yes, if using USB cables between motherboards

> **TIP**
>
> **Correct Answer: [b]** — No, OpenMP is strictly designed for Shared-Memory systems where threads access the same physical motherboard address space
> **Technical Explanation:** Standard OpenMP relies on hardware shared memory (a unified physical address space). It cannot natively communicate across distributed network nodes without hybrid clustering protocols like MPI or distributed shared memory abstractions.

Question 35 [Applied Parallel Concepts & Architecture Preview | Easy | MPI Fundamentals]

Topic: MPI Multi-Process Isolation
Question: When four MPI ranks run on the same physical dual-socket server, how do their memory address spaces interact by default?

- **a)** They share all global variables automatically
- **b)** Each MPI rank is an independent operating system process with its own private virtual memory space; data sharing requires explicit MPI calls or shared-memory IPC
- **c)** They merge into a single thread
- **d)** They can write to each other's stack pointers without restrictions

> **TIP**
>
> **Correct Answer: [b]** — Each MPI rank is an independent operating system process with its own private virtual memory space; data sharing requires explicit MPI calls or shared-memory IPC
> **Technical Explanation:** MPI follows the shared-nothing process model: each rank is an independent OS process with isolated virtual memory. Even on the same motherboard, Rank 0 cannot read Rank 1's variables directly; it must send messages via MPI or shared-memory channels.

Question 36 [Applied Parallel Concepts & Architecture Preview | Medium | HPC Paradigms]

Topic: Hybrid MPI + OpenMP Programming Model
Question: What is the industry-standard 'Hybrid Programming Model' widely deployed on top supercomputers today?

- **a)** Python for front-end, HTML for back-end
- **b)** MPI across distributed nodes (inter-node communication) combined with OpenMP multithreading within each shared-memory node (intra-node core parallelism)
- **c)** JavaScript running inside bash scripts
- **d)** Java Virtual Machines communicating over Bluetooth

> **TIP**
> **Correct Answer: [b]** — MPI across distributed nodes (inter-node communication) combined with OpenMP multithreading within each shared-memory node (intra-node core parallelism)
> **Technical Explanation:** The hybrid MPI+OpenMP model is the gold standard for supercomputing: MPI handles coarse-grained distributed message passing across nodes, while OpenMP handles fine-grained shared-memory multithreading across cores within each node.

Question 37 [Applied Parallel Concepts & Architecture Preview | Medium | Performance Engineering]

Topic: Grain Size in Parallelism
Question: What happens if a parallel algorithm has 'Fine-Grained' parallelism where the computation per task is extremely small compared to thread synchronization overhead?

- **a)** The program achieves linear super-speedup
- **b)** Parallel overhead (thread creation, lock contention, scheduling) exceeds the computational workload, causing the parallel program to run slower than the sequential version
- **c)** The CPU frequency doubles
- **d)** The program crashes with an out-of-memory error

> **TIP**
> **Correct Answer: [b]** — Parallel overhead (thread creation, lock contention, scheduling) exceeds the computational workload, causing the parallel program to run slower than the sequential version
> **Technical Explanation:** If task granularity is too fine, the overhead of creating tasks, acquiring locks, and synchronizing at barriers dwarfs the actual mathematical computation, yielding negative speedup.

Question 38 [Applied Parallel Concepts & Architecture Preview | Easy | Performance Engineering]

Topic: Load Imbalance Definition
Question: In a parallel computing cluster, what is 'Load Imbalance'?

- **a)** When the server rack is physically tilted on the floor
- **b)** When unequal amounts of computational work are assigned to different processors, forcing fast workers to sit idle at barriers waiting for the slowest worker to finish
- **c)** When the power supply provides unbalanced electrical voltage
- **d)** When all cores complete work at the exact same microsecond

> **TIP**
>
> **Correct Answer: [b]** — When unequal amounts of computational work are assigned to different processors, forcing fast workers to sit idle at barriers waiting for the slowest worker to finish
>
> **Technical Explanation:** Load imbalance occurs when work is unevenly distributed across threads or ranks. Because barrier synchronizations force all workers to wait for the slowest task, the slowest worker dictates overall execution time.

Question 39 [Applied Parallel Concepts & Architecture Preview | Medium | Loop Parallelization]

Topic: Data Dependency Hazards (RAW, WAR, WAW)
Question: Which of the following loops CANNOT be safely parallelized with OpenMP `#pragma omp parallel for` without restructuring?

- **a)** `for (int i=0; i<N; i++) C[i] = A[i] + B[i];`
- **b)** `for (int i=1; i<N; i++) A[i] = A[i-1] + B[i];` (Read-After-Write Loop-Carried Dependency)
- **c)** `for (int i=0; i<N; i++) A[i] = 2.0 * i;`
- **d)** `for (int i=0; i<N; i++) C[i] = sin(A[i]);`

> **TIP**
>
> **Correct Answer: [b]** — `for (int i=1; i<N; i++) A[i] = A[i-1] + B[i];` (Read-After-Write Loop-Carried Dependency)
>
> **Technical Explanation:** Loop (b) has a loop-carried dependency (Read-After-Write): calculating `A[i]` requires the result of `A[i-1]` from the preceding iteration. Executing iterations in parallel causes race conditions because iteration i may read uncomputed or stale values.

Question 40 [Applied Parallel Concepts & Architecture Preview | Medium | Empirical Scaling]

Topic: Speedup Saturation & Core Scaling
Question: When running a parallel code across 1, 2, 4, 8, 16, and 32 cores, you notice execution time decreases up to 8 cores, but increases when going from 16 to 32 cores. What is the most common hardware root cause?

- **a)** The CPU ran out of clock cycles
- **b)** Memory Bandwidth Saturation (all cores saturate DDR memory channels) and lock contention/inter-socket NUMA bus overhead outweigh compute gains
- **c)** Linux automatically limits programs to 8 cores
- **d)** The C++ compiler disables optimization past 8 cores

> **TIP**
>
> **Correct Answer: [b]** — Memory Bandwidth Saturation (all cores saturate DDR memory channels) and lock contention/inter-socket NUMA bus overhead outweigh compute gains
>
> **Technical Explanation:** Beyond a certain core count, memory-bound applications hit the memory wall: DDR channels cannot supply data fast enough to feed all cores. Additionally, thread synchronization and cross-socket UPI interconnect traffic create diminishing returns and slowdowns.

Question 41 [Day 2 Advanced Architecture & oneDAL Preview | Easy | oneAPI Ecosystem]

Topic: Intel oneDAL Acceleration Scope
Question: Which family of machine learning algorithms is accelerated by the Intel oneAPI Data Analytics Library (oneDAL)?

- **a)** Deep Convolutional Neural Networks only
- **b)** Classical Machine Learning and Data Analytics (K-Means, SVM, PCA, Linear/Logistic Regression, Random Forest, KNN)
- **c)** Large Language Model text generation exclusively
- **d)** Quantum circuit simulation

> **TIP**
> **Correct Answer: [b]** — Classical Machine Learning and Data Analytics (K-Means, SVM, PCA, Linear/Logistic Regression, Random Forest, KNN)
> **Technical Explanation:** Intel oneDAL focuses specifically on classical machine learning, data mining, and statistical analytics. Deep learning is handled by oneDNN and TensorFlow-MKL, while oneDAL accelerates algorithms like SVM, K-Means, and Random Forest.

Question 42 [Day 2 Advanced Architecture & oneDAL Preview | Easy | oneAPI Software Tools]

Topic: Intel Extension for Scikit-Learn (`sklearnex`)
Question: What is the official package name for the Intel Extension for Scikit-Learn available on PyPI and Conda?

- **a)** `intel-scikit`
- **b)** `scikit-learn-intelex`
- **c)** `fast-sklearn-intel`
- **d)** `mkl-learn`

> **TIP**
> **Correct Answer: [b]** — `scikit-learn-intelex`
> **Technical Explanation:** The official package is `scikit-learn-intelex` (imported as `sklearnex`). It provides seamless drop-in acceleration for Scikit-learn on Intel CPUs and GPUs.

Question 43 [Day 2 Advanced Architecture & oneDAL Preview | Medium | Intel AI Analytics Toolkit]

Topic: Intel Optimized NumPy & BLAS
Question: Which low-level Intel mathematics library powers matrix algebra operations inside Intel-optimized NumPy distributions?

- **a)** Intel oneMKL (Math Kernel Library)
- **b)** OpenCV
- **c)** DirectX
- **d)** FFmpeg

> **TIP**
>
> **Correct Answer: [a]** — Intel oneMKL (Math Kernel Library)
> **Technical Explanation:** Intel-optimized NumPy links directly to Intel oneMKL (oneAPI Math Kernel Library), which provides hand-tuned assembly implementations of BLAS (Basic Linear Algebra Subprograms) and LAPACK leveraging AVX-512 and AMX instructions.

Question 44 [Day 2 Advanced Architecture & oneDAL Preview | Medium | Intel AI Analytics Toolkit]

Topic: Modin: Accelerated Pandas on Multicore
Question: How does Modin (included in the Intel AI Analytics Toolkit) accelerate standard Pandas DataFrame operations?

- **a)** By deleting 50% of the data columns
- **b)** By partitioning DataFrames across both rows and columns and distributing computations seamlessly across all available CPU cores using Ray or Dask backends
- **c)** By converting all numbers into integers
- **d)** By running dataframes on GPU ray tracing cores

> **TIP**
>
> **Correct Answer: [b]** — By partitioning DataFrames across both rows and columns and distributing computations seamlessly across all available CPU cores using Ray or Dask backends
> **Technical Explanation:** Standard Pandas uses a single CPU core. Modin transparently partitions DataFrames across row and column blocks, distributing data processing across all CPU cores without requiring users to rewrite their Pandas code.

Question 45 [Day 2 Advanced Architecture & oneDAL Preview | Easy | Toolchains]

Topic: OpenMP Compilation Flag
Question: Which compiler flag must be passed to `g++` or `icpx` to enable OpenMP directive parsing and link the OpenMP runtime library?

- **a)** `-open-all`
- **b)** `-fopenmp` (for GCC/Clang) or `-qopenmp` (for Intel icx/icpx)
- **c)** `-parallel-mode`
- **d)** `-omp-enable`

> **TIP**
>
> **Correct Answer: [b]** — `-fopenmp` (for GCC/Clang) or `-qopenmp` (for Intel icx/icpx)
> **Technical Explanation:** GNU GCC and Clang use `-fopenmp`. Intel oneAPI compilers (`icx` , `icpx` , `ifx`) use `-qopenmp` (or `-fopenmp`). Without this flag, the compiler ignores `#pragma omp` directives as unrecognized comments.

Question 46 [Day 2 Advanced Architecture & oneDAL Preview | Easy | Toolchains]

Topic: MPI Compilation Wrapper
Question: Which command-line wrapper script is used to compile C++ MPI applications with all required headers and libraries automatically linked?

- **a)** `g++ -mpi`

- **b)** `mpicxx` (or `mpic++`)
- **c)** `compile-mpi`
- **d)** `mpi-build`

> **TIP**
> **Correct Answer: [b]** — `mpicxx` (or `mpic++`)
> **Technical Explanation:** `mpicxx` (or `mpic++` / `mpiicpc`) is a compiler wrapper that automatically includes MPI header directories and links necessary MPI shared libraries (`-lmpi`) before invoking the underlying C++ compiler.

Question 47 [Day 2 Advanced Architecture & oneDAL Preview | Hard | Cache Coherence]

Topic: False Sharing & L1 Cache Invalidation
Question: In OpenMP multithreading, why does an array `int counter[NUM_THREADS]` where each thread `tid` increments `counter[tid]` suffer from severe performance degradation?

- **a)** Because the array exceeds total RAM capacity
- **b)** Because all elements of `counter` sit within the same 64-byte L1 cache line, causing constant false sharing and MESI cache invalidation traffic between CPU cores
- **c)** Because OpenMP cannot index arrays using thread IDs
- **d)** Because C++ integers are not thread-safe

> **TIP**
> **Correct Answer: [b]** — Because all elements of `counter` sit within the same 64-byte L1 cache line, causing constant false sharing and MESI cache invalidation traffic between CPU cores
> **Technical Explanation:** Although threads write to mathematically independent array indices, hardware caches move data in 64-byte chunks. Because 8 integers occupy only 32 bytes, multiple threads write to the same cache line simultaneously, triggering constant MESI cache invalidation storms.

Question 48 [Day 2 Advanced Architecture & oneDAL Preview | Medium | Synchronization Hazards]

Topic: OpenMP Critical Section Overhead
Question: Why should `#pragma omp critical` be placed OUTSIDE high-frequency inner loops whenever possible?

- **a)** Because critical sections disable internet access
- **b)** Because entering and exiting a critical section involves mutex lock acquisitions that serialize all threads, completely eliminating parallel speedup if executed millions of times
- **c)** Because critical sections can only execute 10 times total
- **d)** Because the compiler deletes critical sections during optimization

> **TIP**
> **Correct Answer: [b]** — Because entering and exiting a critical section involves mutex lock acquisitions that serialize all threads, completely eliminating parallel speedup if executed millions of times
> **Technical Explanation:** A critical section serializes threads, forcing $N - 1$ threads to wait while 1 thread executes. Placing it inside a tight loop executed millions of times turns the parallel code into a serialized bottleneck with added locking overhead.

Question 49 [Day 2 Advanced Architecture & oneDAL Preview | Hard | MPI Communication]

Topic: MPI Point-to-Point vs Collective Scalability
Question: Why is replacing a loop of N point-to-point `MPI_Send` calls with a collective `MPI_Bcast` dramatically faster on large HPC clusters?

- **a)** `MPI_Bcast` runs on the motherboard audio card
- **b)** `MPI_Bcast` utilizes hardware-optimized binomial or hypercube tree algorithms with $O(\log P)$ communication steps, whereas a loop of sends takes $O(P)$ sequential steps bottlenecked at Rank 0
- **c)** `MPI_Send` cannot transmit arrays larger than 100 bytes
- **d)** `MPI_Bcast` disables network switches

> **TIP**
> **Correct Answer: [b]** — `MPI_Bcast` utilizes hardware-optimized binomial or hypercube tree algorithms with $O(\log P)$ communication steps, whereas a loop of sends takes $O(P)$ sequential steps bottlenecked at Rank 0
> **Technical Explanation:** A naive loop of sends requires the root rank to send $P - 1$ messages sequentially ($O(P)$ time and linear network injection bandwidth). `MPI_Bcast` uses tree topologies (binomial trees, pipelined rings) that propagate data in $O(\log P)$ steps.

Question 50 [Day 2 Advanced Architecture & oneDAL Preview | Medium | Hardware Acceleration]

Topic: oneDAL Hardware Acceleration Path
Question: When executing `sklearnex.patch_sklearn()`, which hardware instruction set extensions on 4th and 5th Gen Intel Xeon Scalable processors does oneDAL leverage for maximum throughput?

- **a)** ARM Neon instructions
- **b)** Intel AVX-512 vector extensions and Intel AMX (Advanced Matrix Extensions)
- **c)** Legacy MMX 64-bit integer registers
- **d)** 8086 16-bit real mode registers

> **TIP**
> **Correct Answer: [b]** — Intel AVX-512 vector extensions and Intel AMX (Advanced Matrix Extensions)
> **Technical Explanation:** Intel oneDAL includes dispatchers that detect CPU capabilities at runtime. On modern Xeon Scalable processors, it activates AVX-512 FMA vector units and Intel AMX (Tile Matrix Multiply) units to execute machine learning algorithms at peak silicon performance.
